

AegisCare HMS / LabTrack v5.5

System name	AegisCare HMS / LabTrack
Version	5.5
Repository	fmurage6331-dev/confit-core
Live URL	https://aegiscarehms.lovable.app
Supabase project	tgynjasgnerucrlwedui
Document version	1.0 (draft for DHA certification submission)
Date	2026-08-12

Table of Contents

1. System Purpose & Scope
2. Architecture Overview
3. Technology Stack
4. Database Schema Summary
5. User Roles & Access Control
6. Clinical Workflows Supported
7. DHA HIE Integration Architecture
8. SHA Claims Workflow
9. External Integrations
10. Deployment & Infrastructure

Section 1 — System Purpose & Scope

AegisCare HMS is a hospital management system for Kenyan healthcare facilities. It provides the complete operational record of care at FACILITY_NAME (FACILITY_ADDRESS; KMHFL code FACILITY_KMHFL_CODE): patient registration, queue and room-based routing, triage and vitals, consultation with structured ICD-11 diagnosis, laboratory and radiology order/result workflows, inpatient wards and beds, surgical theatre, mortuary, maternal and child health (MCH) and family planning, pharmacy dispensing with stock control, billing and invoicing, insurance and SHA (Social Health Authority) claims management, MOH national reporting, and administrative/security functions.

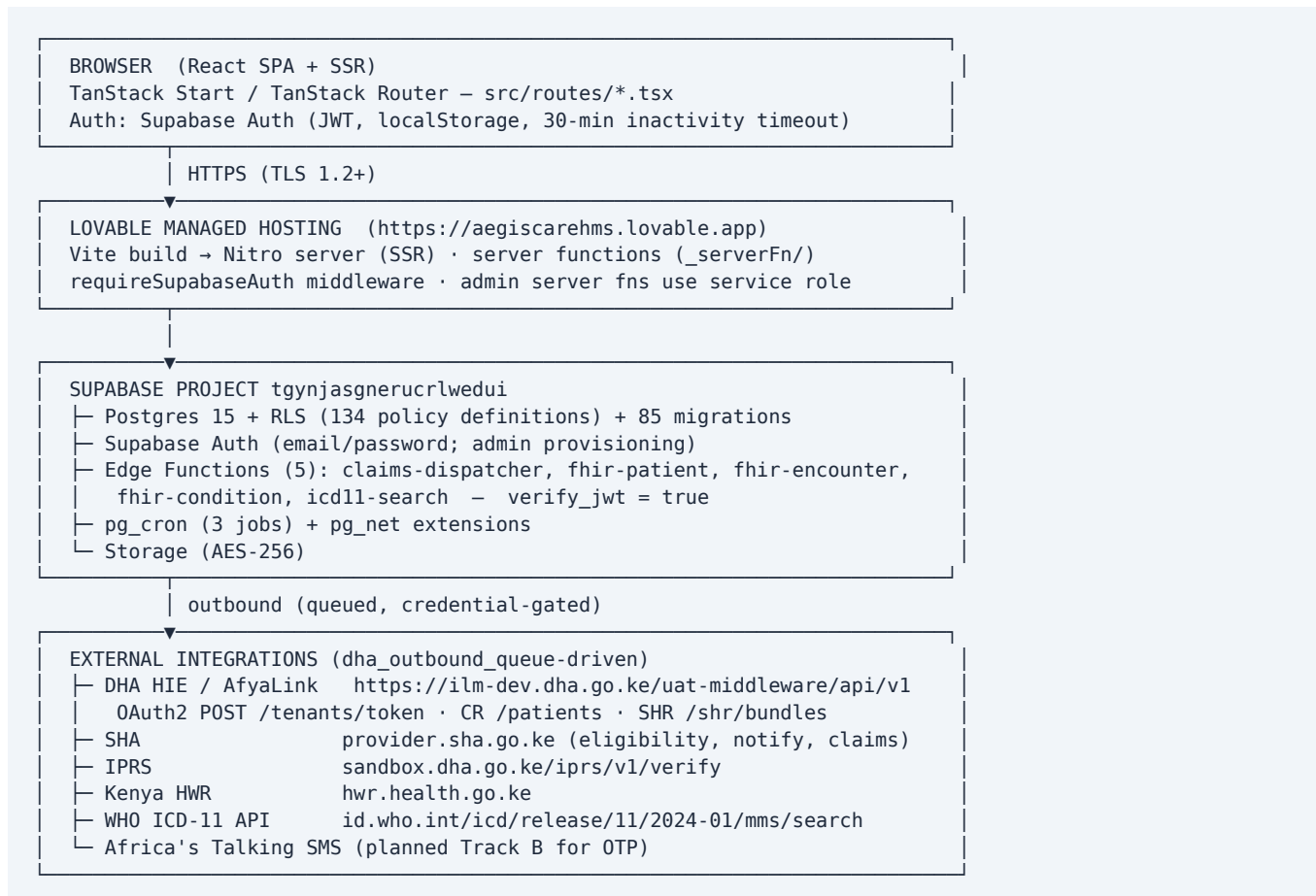
The system is designed to integrate with Kenya's national digital health ecosystem under the Digital Health Act, 2023:

- DHA Health Information Exchange (HIE) — FHIR R4 resource exchange with the Shared Health Record (AfyaLink) (dha_outbound_queue, FHIR edge functions);
- Client Registry — national patient identity resolution (CR ID);
- SHA — benefit packages, eligibility, preauthorisation, notification numbers and electronic claims;
- NLMIS / KEMSA — commodity coding of pharmaceutical stock;
- Kenya HWR — practitioner council verification (KMPDC / NCK / KMLTTB);

- MOH reporting — forms 204, 505, 642, 705, 706, 707, 717, FP, MCH.

Deployment model: one isolated deployment per facility (Model A) — each facility has its own repository fork, its own Supabase project, and its own Lovable deployment, so no facility can observe another facility's data (docs/facility-onboarding.md).

Section 2 — Architecture Overview



2.1 Frontend

- React 19 + TanStack Start (SSR) + TanStack Router (file-based routing under `src/routes/`), Tailwind CSS 4 + shadcn/ui components (`src/components/ui/`), TanStack Query for server state, Zod validation on forms.
- The single most important clinical route is `src/routes/rooms.$id.tsx` (5,685 lines), which dynamically serves all room-kind workflows: triage, consultation, mch, pharmacy, lab, radiology, billing, insurance, ward, theatre, mortuary — one route file, not separate files per department (intentional architecture).
- Auth state, roles and permissions are managed in `src/lib/auth-context.tsx` (AuthProvider, useAuth, hasPerm); access guards in `src/lib/require-access.tsx` (Guard, PermGuard).

2.2 Backend

- Supabase (PostgREST) with Row-Level Security as the enforcement layer.
- Server functions (createServerFn in `src/lib/admin-users.functions.ts`) run on the Nitro SSR server behind `requireSupabaseAuth` (`src/integrations/supabase/auth-middleware.ts`); the Supabase admin (service-role) client exists only server-side (`client.server.ts`).

- 5 Deno edge functions under supabase/functions/, all with `verify_jwt = true` (supabase/config.toml), deployed via GitHub Actions (.github/workflows/deploy-functions.yml → project tgynjasgnerucrlwedui).

2.3 Database

- PostgreSQL 15 with 85 migration files in supabase/migrations/ (the authoritative schema), ~64 core tables, 3 views (patient_registrations, encounter_records_summary, stock_store_balances_view, stock_store_usage_view, moh_705_* related views), 57 trigger definitions, 134 RLS policy statements, ~70 PL/pgSQL functions, 3 pg_cron jobs, 2 extensions (pg_cron, pg_net).

2.4 Edge functions (summary)

Function	Purpose	Status
claims-dispatcher	Routes SHA/private/cash/FHIR events into the_outbound_queue	Live, stub mode (queues only)
fhir-patient	FHIR R4 Patient resource from patients	Live
fhir-encounter	FHIR R4 Encounter resource from encounters	Live
fhir-condition	FHIR R4 Condition[] from encounter_diagnoses	Live
icd11-search	Proxy to WHO ICD-11 MMS search (token cached)	Live, needs ICD_CLIENT_ID/ICD_CLIENT_SECRET; falls back to local icd11_codes

2.5 Hosting & external integrations

See Sections 9 and 10.

Section 3 — Technology Stack

Layer	Technology	Evidence
Frontend	React 19.2, TanStack Start 1.167.x (SSR), TanStack Router 1.168.x, TanStack Query 5.83	package.json
Styling/UI	Tailwind CSS 4.2, shadcn/ui (Radix primitives), lucide-react, recharts, sonner, vault, cmdk, input-otp	package.json, components.json
Forms/validation	react-hook-form 7.71, zod 4.4	package.json
Backend	Supabase JS 2.110, PostgREST, Deno edge functions	src/integrations/supabase/client.ts
Database	PostgreSQL (Supabase), RLS, pg_cron, pg_net	supabase/migrations/
Runtime/build	Bun (CI: bun install --frozen-lockfile), Vite 7.3, Nitro 3 beta, TypeScript 5.8	package.json, .github/workflows/ci.yml
Deployment	Lovable managed hosting (Vite/TanStack config via Deployment@lovable.dev/vite-tanstack-config), GitHub Actions, Supabase Cloud	vite.config.ts, .github/workflows/
Security patches	seroval pinned ^1.5.4 (CVE-2026-23737); xlsx removed (CVE-2023-30533, CVE-2024-22363); patch-package postinstall	package.json overrides, patches/
Standards	FHIR R4, ICD-11 (MMS linearization 2024-01), SHA benefit packages, NLMIS codes	Edge functions, migrations, docs/

Section 4 — Database Schema Summary

All objects below are defined across the 85 files in supabase/migrations/.
Views are marked *(view)*.

Table	Purpose
patients	Demographics; national_id, national_id_type, sha_member_number, sha_membership_status, cr_number (Client Registry), blood_group, allergies,
encounters	Core visit record: payment, tests JSONB, vitals, history, diagnoses JSONB, room routing, is_emergency, referral fields, SHA columns (sha_notification_number, insurer_type, claim_number, claim_status, preauth_number, sha_fund_type)
patient_registrations (view)	Encounters × patients join used by registration UI; INSTEAD OF triggers patient_registrations_insert_trg / patient_registrations_update_trg
patient_registrations_legacy	Pre-view legacy table (retained for history)
admissions	Inpatient admissions: bed, ward, admitting doctor, discharge
beds / wards	Bed occupancy and ward definitions (daily_rate drives bed-charge accrual)
encounter_room_visits	Room movement trail (routing, MOH tagging)
encounter_diagnoses	Structured ICD-11 diagnoses: icd11_code, icd11_title, icd11_uri, diagnosis_type (primary/secondary/differential/final/working/admission/discharge), sequence
encounter_indicator_tags	MOH indicator tags per encounter (OPD_UNDER5_M, LAB_HIV, ...)
clinical_notes	Doctor notes, discharge summaries (note_type)
lab_orders / lab_results	Order-driven laboratory worklist; specimen_type, is_critical, verified_by/at dual verification
lab_tests	Legacy laboratory records (pre-worklist)
lab_test_catalog	Test/service catalogue with kind (lab/service/radiology), cash & insurance pricing, target_room_id
radiology_orders / radiology_results	Radiology order/result lifecycle
prescriptions	Drug orders: drug_name, dosage, frequency, quantity, status pending/dispensed/cancelled, dispensed_by/at
medication_administrations	MAR — nurse-recorded administrations for inpatient prescriptions
stock_items	Inventory: nlmis_code, current_quantity, reorder_level, pricing, strength
stock_movements	Stock ledger: change, reason (delivery/dispense/usage/adjustment/stock_take)
deliveries	Goods received (auto stock movement via trg_deliv_stock)
machines / machine_logs	Equipment registry and maintenance/calibration/service logs
invoices / invoice_line_items / invoice_payments	Billing: INV-YYYY-##### numbering, itemised lines, payments, totals recalculation triggers
insurance_providers	Insurer registry: coverage_percentage, coverage_rule (percentage/fixed_per_visit/percentage_with_cap), per_visit_limit
contracted_prices	Per-insurer rate cards (item_type: lab_test/stock_item/ward)
sha_tariffs	18 seeded SHA tariffs (Legal Notices 146/147), fund-type scoped
sha_benefit_packages	22 SHA benefit packages (PHF 3 / SHIF 15 / ECCIF 4), requires_preauth, limits, combination matrix
sha_claims	SHA/private claims lifecycle (draft→...→paid), OTP and preauth evidence, fhir_bundle
sha_claim_items / sha_claim_packages	Claim line items and package assignment

Table	Purpose
dha_outbound_queue	Outbound integration queue: queue_type (fhir_sync/sha_claim/private_claim/cash_receipt/shr_access_notification), status, attempts, response
shr_transmission_log	Statutory SHR transmission metadata log (append-only)
consent_otps	OTP records (SHA-256 hash only), 10-min expiry, consent_type
patient_consent	Consent ledger (live table; hie_data_sharing_consented gates FHIR sync)
audit_log / audit_log_archive / audit_archive_runs	Append-only audit trail, archive and run log
access_requests	User access-request queue
user_roles / role_permissions / user_room_access	RBAC: 19-role assignment, permissions, room-level grants
profiles	Username/names per auth user + council registration fields
app_settings	Facility configuration (single row id='global'): facility name, KMHFL code, SHA ID, provider no., county, address, phone, email, facility_level
appointments	Scheduled visits (time_range tstzrange)
mortuary_records	Body intake (internal/external), storage charges, release
moh_705_disease_mappings	MOH 705 disease ↔ ICD-11 chapter mapping
moh_indicator_definitions / moh_indicators / moh_monthly_aggregates	MOH indicator engine
moh_report_templates / moh_report_line_items / moh_report_submissions / moh_report_submissions	MOH report workbook
icd11_codes	Local ICD-11 reference table (primary working source; WHO API fallback when configured)
rooms / room_indicator_map	Room registry with kind (general/lab/radiology/triage/consultation/pharmacy/billing/insurance) and MOH
encounter_amendments	Signed-encounter amendment trail
test_templates	Lab parameter templates
fund_utilizations	Student/staff/external fund usage

Section 5 — User Roles & Access Control

All 19 roles are defined in `src/lib/roles.ts` and the `public.app_role` enum (20260716063555... + 20260730140000_dha_task_d_security_roles.sql), enforced through `is_approved()` at the RLS layer and `user_has_permission()` / `role_permissions` at the feature layer.

#	Role	Typical duties
1	admin	Facility administrator — users, rooms, pricing, settings, all data
2	system_admin	Platform-level administration
3	staff	General staff (legacy broad role)
4	receptionist	Registration, queueing, appointments
5	accountant	Billing, payments, waivers
6	insurance_agent	SHA/insurance desk, claims
7	records_officer	Records, MOH reports
8	triage_nurse	Vitals, triage

#	Role	Typical duties
9	nurse	Ward care, medication administration
10	doctor	Consultation, diagnosis, prescribing, sign_encounter
11	clinical_officer	Consultation, diagnosis, sign_encounter
12	dental_officer	Dental consultation, sign_encounter
13	nutritionist	Nutrition services
14	physiotherapist	Physiotherapy services
15	hts_counsellor	HIV testing & counselling
16	lab_tech	Laboratory orders/results
17	radiologist	Radiology orders/results
18	pharmacist	Dispensing, stock
19	mortician	Mortuary records

Access-control primitives:

- `is_approved(auth.uid())` — RLS gate (`SECURITY DEFINER`, `search_path=public`).
- `has_role(_user_id, app_role)` and `user_has_permission(_user, perm)`.
- Room-level access: `user_room_access` + `can_access_room()`.
- UI guards: `useHasAccess` / `useHasPerm` / `Guard` / `PermGuard` (`src/lib/require-access.tsx`), `hasPerm` (`src/lib/auth-context.tsx`).
- Admin-only user lifecycle: `src/lib/admin-users.functions.ts` (server fn, `assertAdmin`).

Section 6 — Clinical Workflows Supported

All routes under `src/routes/` (TanStack file-based routing).

Workflow	Route(s)	Notes
Registration	<code>register-patient.tsx</code>	Demographics, insurance/SHA, IPRS verify button, OTP consent
Queue	<code>queue.tsx</code>	Live waiting queue
Triage	<code>rooms.\$id.tsx (kind=triage)</code>	Vitals/anthropometrics
Consultation	<code>rooms.\$id.tsx (kind=consultation/mch)</code>	History, ICD-11 diagnosis (local <code>icd11_codes</code> + <code>icd11-search</code>), prescriptions, tests
Laboratory	<code>laboratory.index.tsx</code> , <code>laboratory.\$id.tsx</code>	Worklist, specimen capture, results, critical + dual verify
Radiology	<code>radiology.index.tsx</code> , <code>radiology.\$id.tsx</code>	Orders, image paths, reports
Pharmacy	<code>rooms.\$id.tsx (kind=pharmacy)</code>	Dispensing with stock guard; MAR on wards
Insurance desk	<code>rooms.\$id.tsx (kind=insurance)</code> , <code>admin.insurance.tsx</code> , <code>admin.queue.tsx</code>	Benefit packages, preauth, claims, queue monitor
Billing / accounting	<code>accounting.tsx</code> , <code>rooms.\$id.tsx (kind=billing)</code> , <code>invoices.index.tsx</code> , <code>invoices.\$id.tsx</code>	Payments, waivers, invoices

Workflow	Route(s)	Notes
Inpatient	inpatient.tsx, inpatient_.\$admissionId.tsx, admin.wards.tsx, rooms.\$id.tsx (kind=ward)	Admissions, bed management, IPD chart (notes, lab, radiology, MAR), discharge summary gate
Theatre	rooms.\$id.tsx (kind=theatre)	Surgical cases
Mortuary	rooms.\$id.tsx (kind=mortuary)	Intake, storage accrual, release
MCH / FP	rooms.\$id.tsx (kind=mch), moh.mch.tsx, moh.fp.tsx	MCH/FP indicators
Records	records.index.tsx, records.\$id.tsx, records.new.tsx, encounter-records.index.tsx, encounter-records.\$id.tsx	Legacy + encounter records, signing
Appointments	appointments.tsx	Scheduling
Stock	stock.tsx, deliveries.tsx, machines.tsx	Inventory, deliveries, equipment
MOH	moh.index.tsx, moh.204.tsx, moh.505.tsx, moh.642.tsx, moh.705.tsx, moh.706.tsx, moh.707.tsx, moh.717.tsx, moh.fp.tsx, moh.mch.tsx	Auto-tagged indicators → monthly aggregates
Reports	reports.tsx, dashboard.tsx	Aggregates, dashboard stats
Admin	admin.users.tsx, admin.permissions.tsx, admin.queue.tsx, admin.audit-log.tsx, admin.settings.tsx, admin.pricing.tsx, admin.services.tsx, admin.rooms.tsx, admin.wards.tsx, admin.test-templates.tsx, admin.moh-indicators.tsx, admin.amendments.tsx	Full administration
Auth	login.tsx, forgot-password.tsx, reset-password.tsx, change-password.tsx, account.tsx	Auth + profile + practitioner verification

Encounter signing (src/routes/rooms.\$id.tsx ~line 949): users with `sign_encounter` permission set `encounters.status = 'signed'`; `enforce_encounter_lock()` then blocks further INSERT/UPDATE/DELETE on `encounter_diagnoses`, `clinical_notes` and `radiology_orders` for signed/finalized/completed encounters (Sprint 13A, 20260811000002_sprint13a_encounter_signing.sql); amendments are tracked in `encounter_amendments`.

Section 7 — DHA HIE Integration Architecture

7.1 Resource generation (database layer)

- `generate_fhir_encounter(p_encounter_id UUID)` — STABLE, SECURITY DEFINER, `search_path=public`; emits FHIR R4 Encounter incl. ICD-11 diagnoses (rank/role), SHA notification identifier, fund-type `serviceType`, KMHFL `serviceProvider`, practitioner with council identifier, referrals, blood group, allergies, clinical findings extension (Second Schedule coverage).
- `trg_medication_dispense_fhir()` — FHIR R4 MedicationDispense on dispense.
- `encounter_diagnoses` ↔ FHIR Condition mapping via `fhir-condition`.

7.2 Edge-function layer (HTTP)

- `fhir-patient`, `fhir-encounter`, `fhir-condition` — on-demand FHIR resources (application/fhir+json), JWT-verified, service-role DB access.
- `claims-dispatcher` — orchestrates outbound events: `FhirSyncHandler` (consent-gated), `ShaClaimsHandler`, `PrivateClaimsHandler`, `CashReceiptHandler` → `dha_outbound_queue`.

7.3 Outbound queue

- `dha_outbound_queue` (20260812000006_drift_dha_outbound_queue.sql):
queue_type CHECK (fhir_sync | sha_claim | private_claim | cash_receipt | shr_access_notification), status machine (pending → processing → sent → acknowledged | failed | skipped), attempts, response, error_message.
- Every status transition for fhir_sync/shr_access_notification is mirrored to shr_transmission_log (statutory metadata log).
- The queue is the single integration seam: when DHA/SHA credentials arrive, only the dispatcher handlers change (stub → live), per docs/integration-activation-manual.md.

7.4 Consent gating

- `patient_consent.hie_data_sharing_consented = true` required before any FHIR sync is queued (checked in FhirSyncHandler and captured in ConsentDialog).

Section 8 — SHA Claims Workflow

End-to-end (all steps implemented and live except where noted):

1. Registration — receptionist captures SHA member number (`patients.sha_member_number`), relationship to principal, and records OTP-verified consent (`consent_otps + patient_consent`, SHA Act 2023 s.48).
□ PENDING: live member verification via SHA API.
2. Visit initiation — `encounters.sha_notification_number` captured at the insurance desk; currently TEMP-SHA-`{ts}-{id}` placeholder. □ PENDING: real notification numbers from SHA eligibility API.
3. Fund classification — `set_sha_fund_type()` auto-assigns phf / shif / eccif (emergency→ECCIF, inpatient→SHIF, else PHF); overridable at the insurance desk.
4. Service delivery — consultation with ICD-11 diagnoses (`encounter_diagnoses`, sequence-ranked), lab/radiology/pharmacy with invoice line items.
5. Signing — clinician (`sign_encounter`) signs the encounter (`status = 'signed'`); record locks; `trg_auto_generate_sha_claim` fires and creates a draft `sha_claims` row + line items from `invoice_line_items` (excluding `credit_note`).
6. Insurance desk — selects SHA benefit packages (`sha_benefit_packages`, 22 packages) and claim line items (`is_included`), enforces preauth for `requires_preauth` packages, sets `status = 'ready'`, updates totals.
7. Submission — `submitClaim()` writes `dha_outbound_queue` (`queue_type='sha_claim'`, payload includes `fhir_encounter`) and marks the encounter `claim_status = 'submitted'`. □ PENDING: live POST to SHA claims API (stub `ShaClaimsHandler`).
8. Tracking — `admin.queue.tsx` claims aging monitor; claim lifecycle statuses (`submitted → acknowledged → approved | rejected → appealed → paid`) recorded on `sha_claims` and `encounters`.

Section 9 — External Integrations

Integration	Live?	Built?	Evidence
DHA HIE / AfyaLink (FHIR)	pending credentials	resources + queue + consent gate	generate_fhir_encounter(), FHIR edge functions, dha_outbound_queue
Client Registry (IPRS)	pending credentials	schema + UI + stub RPC	patients.cr_number, verify_patient_identity(), register-patient.tsx
SHA member registry	pending credentials	schema + UI	patients.sha_membership_status, badge on profile/accounting/insurance
SHA notification numbers	pending credentials	schema + capture	encounters.sha_notification_number (TEMP placeholder)
SHA claims API	pending credentials	stub handler + queue	claims-dispatcher ShaClaimsHandler
Preauthorisation API	pending credentials	local capture/enforcement	sha_benefit_packages.requires_preauth, insurance desk
Kenya HWR practitioners	pending credentials	schema + stub RPC	profiles.council_*, verify_practitioner(), account.tsx
WHO ICD-11 API	needs	icd11-search (auto-fallback to ICD-10 codes)	supabase/functions/icd11-search/index.ts
Africa's Talking SMS (OTP)	pending credentials	Track A on-screen OTP live; Track B stubbed	consent-dialog.tsx, docs/integration-activation-manual.md
M-Pesa Daraja	planned	not built	docs/integration-activation-manual.md Integration 8
MOH reporting (KHIS)	internal generation; manual submission	9 forms + aggregates	src/routes/moh.*.tsx, refresh_moh_aggregates()

Section 10 — Deployment & Infrastructure

- CI/CD: GitHub Actions — ci.yml (lint + test + build on Bun with --frozen-lockfile) and deploy-functions.yml (deploys edge functions to tginjasgnerucrlwedu on main push when supabase/functions/ changes).
- Hosting: Lovable managed (SSR via Nitro; preview/production URLs). Environment: VITE_SUPABASE_URL, VITE_SUPABASE_PUBLISHABLE_KEY (publishable-key API), server-side SUPABASE_URL / SUPABASE_PUBLISHABLE_KEY for SSR auth middleware.
- Database: Supabase project tginjasgnerucrlwedu; schema managed exclusively by the 85 migration files (apply in filename order per docs/facility-onboarding.md); extensions pg_cron, pg_net; backups and encryption per Supabase platform (see DOC-4).
- Edge functions: deployed with supabase functions deploy <name> --project-ref tginjasgnerucrlwedu; secrets stored in Supabase Edge Function secrets only (never in the repository).
- Facility onboarding: one deployment per facility — fork, create Supabase project, run migrations, connect Lovable, configure facility details in Admin → Settings → Facility tab, set secrets, create admin user (docs/facility-onboarding.md).

End of DOC-1.